This CVPR workshop paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Interpolation-Based Event Visual Data Filtering Algorithms

Marcin Kowalczyk, Tomasz Kryjak
Embedded Vision Systems Group, AGH University of Krakow
kowalczyk@agh.edu.pl tomasz.kryjak@agh.edu.pl

Abstract

The field of neuromorphic vision is developing rapidly, and event cameras are finding their way into more and more applications. However, the data stream from these sensors is characterised by significant noise. In this paper, we propose a method for event data that is capable of removing approximately 99% of noise while preserving the majority of the valid signal. We have proposed four algorithms based on the matrix of infinite impulse response (IIR) filters method. We compared them on several event datasets that were further modified by adding artificially generated noise and noise recorded with dynamic vision sensor. The proposed methods use about 30KB of memory for a sensor with a resolution of 1280 × 720 and is therefore well suited for implementation in embedded devices.

1. Introduction

The field of computer vision has been developing rapidly for a long time. One of the areas that has become particularly popular in recent years is neuromorphic vision. Dynamic vision sensors respond to changes in pixel brightness, rather than registering the absolute value of the intensity and colour of the light falling on the matrix pixels. Cameras operating in this way have a number of advantages over traditional solutions (hereafter referred to as frame cameras). The first is high temporal resolution. Changes are monitored at a frequency of up to 1 MHz (depending on the sensor model). This makes it possible to monitor very fast movements with very small motion blur (depends on the illumination of the observed scene [9]), which is a common problem with frame cameras. The second benefit is low latency. Thanks to the independent operation of the pixels, there is no need to wait for the entire image to be processed. This allows to achieve real-world latencies in the order of about one to a few ms [9]. The third is low power consumption, as only brightness change information is transmitted, eliminating data redundancy. The power consumption of the sensor itself is up to a few hundred mW [5]. A final advantage is the high dynamic range. In the case of event cameras, this can even exceed 120 dB. This is considerably more than even high-quality conventional cameras, which have a dynamic range of around 60 dB. This allows operation in both very dark and very bright environments (even within the same scene).

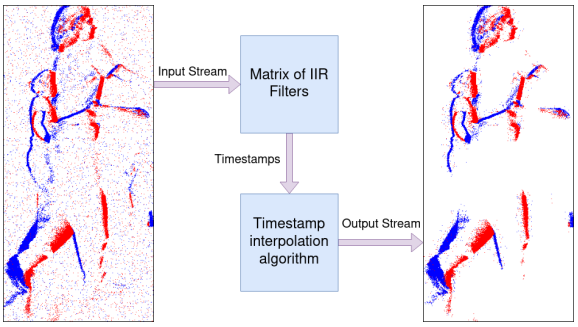


Figure 1. A simplified diagram of the proposed method and an example of input and output data, aggregated for 20 ms.

The use of event cameras also presents challenges with a new type of vision information. The data obtained is a sparse point cloud in three-dimensional space (pixel coordinates, time of occurrence and polarity). Therefore, well-known image processing algorithms developed over several decades cannot be directly applied to such data. Information about the intensity and colour of the light falling on a given pixel is also not available. Only information about the increase or decrease in brightness of a given receptor on the matrix is obtained. Due to the non-ideality of the sensors, photon shot noise and noise from the analogue part of the pixel array, these sensors are very prone to noise. These appear as events that are not related to the movement of the scene or changes in illumination. They resemble impulse noise, which is very easy to see in the output data. It is worth noting that the presence of noise in the processed data is a potential source of reduced algorithm performance.

Figure 1 shows a simplified schematic of the proposed method, with example input and output data. The events have been accumulated over a period of 20 ms. A large reduction in non-motion related noise is evident.

In this paper we propose 4 algorithms for event data fil-

tering based on the solution presented in the paper [12]. The aim of the research described in this article was to develop an algorithm that requires a small amount of memory and at the same time has a high interference filtering efficiency, with as few correct events removed as possible. In order to reduce the number of valid events removed, the proposed algorithms should reduce the negative effects associated with the transition of observed objects between neighbouring regions, which was present in the original algorithm. This type of solution is well suited for implementation in systems with a small amount of memory, or when excessive memory usage negatively affects performance (e.g. the processor is equipped with small cache resources). The proposed methods have a high filtering efficiency for uncorrelated interference, comparable to popular filtering algorithms, such as the nearest neighbour method (NNb).

The rest of the article is structured as follows. Chapter 2 describes the research published so far on event sensor data filtering. Section 3 presents the basic filtering algorithm developed in previous research. The proposed filtering methods are then discussed in section 4. Their evaluation and comparison is presented in section 5. The article concludes with a summary and possible further development of the ongoing research in section 6.

2. Previous work

The topic of video filtering of event data has been addressed many times in the scientific literature due to its great practical importance. This subsection presents an analysis and comparison of different approaches to this problem.

A 2008 paper [4] presented one of the first event stream filtering methods. It consisted of removing events for which no activity was recorded in their surroundings within a given time window. This allowed the removal of single, isolated events. The requirement was to store a timestamp map with a size equal to the sensor resolution multiplied by 2. In this paper, a sensor with a resolution of 128×128 was used.

The filtering of event data with greyscale information is presented in the article [10]. An asynchronous time-based image sensor (ATIS) with a resolution of 304×240 was used. Such sensors report events when the brightness of the pixels changes, but when an event is detected for a particular pixel, a reading of the incident light intensity is initiated. As a result, the greyscale value of the reporting pixel is also available for each event. Asynchronous linear and non-linear filtering techniques have been proposed. These include the Gaussian blur filter, the bilateral filter and the Canny edge detector.

A framework for removing uncorrelated noise on an FPGA platform was presented in the paper [13]. Timestamps corresponding to the processed events were stored in a register matrix. For each input event, the previously stored timestamp was read from the matrix. The difference between the input one and the read one was then calculated. If this was greater than a preset threshold, the event was erased. A new timestamp was written to the same location in the array. In the work in question, this had a size of 128×128. The work also considered writing the received timestamp to neighbouring pixels.

In the paper [14], a background activity filter was proposed to pass on space-time correlated events for further processing. The aim was to reduce communication and computational overhead while increasing the rate of correct information content. A 128×128 matrix chip with 20×20 μm^2 cells was designed. Each cell combines subsampling in the spatial domain with a time window based on current integration.

The paper [3] compares eight filtering algorithms. Three methods calculate the difference in timestamps between the pixel being processed and the last event in its vicinity. The fourth requires that at least two other events belong to the neighbourhood of the processed event within the given time window. Otherwise it will be deleted. The next one deletes the data if its polarity is the same as the previous event reported by the pixel. The next two methods erase the data if not enough time has elapsed since the previous event reported by the pixel in question. The last method calculates the average time between events reported by all pixels.

The article [1] introduces the LDSI algorithm (Less Data Same Information). Its aim is to reduce the amount of data processed without removing relevant information. It used impulse cells, inspired by the action of biological neurons, to process data in four layers. Successive events increased the potential of the corresponding neurons and their neighbours in the third layer.

In the paper [11], a spatiotemporal filter with O(N) memory complexity was presented. In this method, instead of using one memory cell for each pixel, it was proposed to use two cells for each row and column, which allowed a significant reduction in memory usage. An implementation in an FPGA chip is also presented.

An implementation of a filtering algorithm using a pulse neural network in the IBM TrueNorth Neurosynaptic System neuromorphic processor is presented in the paper [17]. A variation of the integrate-and-fire neuron model was used. The network has two layers. The first introduces a break period. It reduces the maximum response frequency of the neurons to remove high frequency noise from a single pixel. The second layer is a neural implementation of the NNb method. It checks whether other events have been generated in the vicinity of the event being processed.

In the paper [2], a method for event stream filtering and compression based on two switched time windows was presented. These windows operate alternately in two phases. In the first, events are input into one window and the data

from the other window are processed and cleaned. In this way, an image representing the sensor data is created. In the second phase, the operation of the windows is reversed. The data from the first window is read and cleaned, and the data is written to the second window. However, as a result of the algorithm, the time stamps of individual events are removed and are not passed to the filter output.

A pedestrian detection system is presented in [16]. In its first part, a point process filter is proposed to filter out noise in the event data. It uses an adaptive time window. It allowed to increase the classification accuracy. In the second part, a binary neural network was used for classification. The proposed filtering method outputs a two-dimensional image in which the recorded events are marked. They are aggregated within a given time window (3 ms). In the generated image, the logical product of adjacent pixels is performed (separately in the horizontal and vertical directions).

The authors of the paper [7] proposed a hashing-based filtering algorithm that does not require the storage of 32-bit timestamps, thus achieving low memory requirements. It was also implemented on an FPGA chip.

An impulse neural network was also presented in the paper [18]. It had an adaptive time window length. A leaky integrate-and-fire (LIF) neuron model was used. According to the authors, the proposed method performs better than the classical filtering method with a time window.

Filtering inspired by the action of the human retina has been proposed in [8]. A retinal model based on Gaussian difference filters inspired by the fovea of the optic nerve was used. A sensor with a resolution of 128 $\times$ 128 was used in this work.

An adaptive background activity filtering method for event data was presented in [15]. The authors used an optical flow and a non-parametric KNN (K-nearest neighbour) regression algorithm. In the first stage of the algorithm, redundant events are removed using a technique based on a dynamic timestamp. In the second stage, background activity noise is removed using the adaptive KNN algorithm.

The paper [6] proposes 3 algorithms for filtering event data. The first calculates the distance to previous events stored in the first-in-first-out memory. The second checks whether a certain number of other events have been registered in the vicinity of the processed event within a given time window. The last is based on a simple neural network that processes the age of events from the analysed neighbourhood.

The literature review confirms the importance and popularity of event data filtering. Newer and newer algorithms are proposed to remove distortions.

3. Basic version of the algorithm

In the paper [12], a filtering method based on an infinite impulse response filter array was proposed. The proposed filtering method is able to remove more than 99% of the uncorrelated noise. The operation of the algorithm is based on dividing the event camera matrix into regions. Each event is passed to a filter corresponding to the area of the event being processed. Each domain contains a timestamp. An event is removed if the difference between the internal area marker and the timestamp of the processed event is greater than a preset threshold. The filtering algorithms proposed in this paper are an extension of this method.

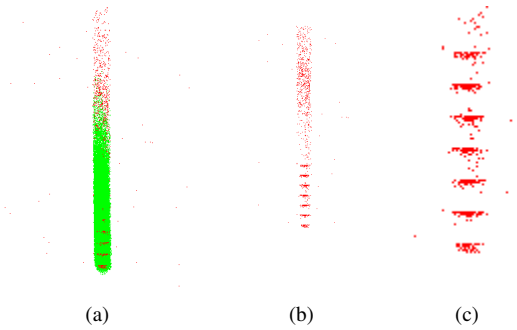


Figure 2. Example of a filter result from the paper [12] for a test sequence. In figure 2a the deleted events are shown in red and the passed events are shown in green. 2b shows only filtered events. 2c shows a close-up on the events removed due to the object's transition between neighbouring regions.

Its greatest feature was the complete independence of operation of the areas into which the sensor matrix was divided. This could be seen by observing the performance of the algorithm for objects moving within the field of view of the event sensor. Correct events were removed when an object moved between adjacent areas. This is because the area in question had not previously received any events, so its timestamp is too small for a new event to pass through. Figure 2a shows the result of filtering for a test sequence containing a recording of falling corn kernels. Events with a period of 10 ms are marked in the figure. The red colour indicates the events removed by the above filter, while the green colour indicates those passed to its output. The filter parameters were as follows: filter length – 400 µs, area size – 16 $\times$ 16, update factor – 0.25.

Figure 2b shows only events filtered out by the same filter. At the bottom of the graph, groups of rejected events are visible. They form below the lines that mark the edges between neighbouring regions. A close-up of these groups is shown in figure 2c. This is due to the phenomenon mentioned earlier - the timestamp of the new region is too small for new events to pass through for further processing.

4. Interpolation-based algorithm

The aim of this research was to reduce the effect described in section 3 by using the parameters of neighbouring areas. The possibility of using algorithms to interpolate

the timestamps of neighbouring areas to smooth the effect of objects passing between neighbouring areas was investigated. Several possibilities for combining these data were considered and their effectiveness analysed. The proposed methods used: timestamps of neighbouring areas, distances of events from neighbouring areas, frequencies of events in neighbouring areas. Four algorithms were tested on the data: maximum timestamps, bilinear interpolation, bilinear interpolation with frequency weights, and distance-based interpolation with frequency weights. A common part of all these algorithms is the estimation of the parameters of the areas into which the event sensor matrix has been divided.

The parameters of the algorithms developed are: sensor resolution, filter length – the threshold that determines which events to filter out, scale – the size of the areas into which the matrix is divided, update rate – the rate at which the parameters of the areas are updated. The transmitted events are processed one by one. Each event is first checked to see if it should be filtered out or sent for further processing. Then the parameters of the area to which the processed event belongs are updated.

The first parameter to be determined is the area's timestamp, which is the filtered timestamps of events in the area. Filtering makes it possible to retain some information about previous events in a given environment. As opposed to storing only the timestamp of the most recent event, this filtering makes it possible to achieve much greater resistance to high-intensity disturbances. The formula for calculating the timestamp is given in the equation (1).

$$Ts_{n+1} = Ts_n \cdot (1 - u) + Ts_e \cdot u \tag{1}$$

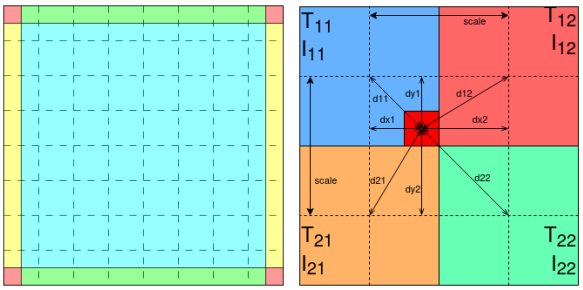
where: Ts_n is the current timestamp of the region, Ts_{n+1} is the new timestamp of the region, u is the update factor, Ts_e is the timestamp of the processed event.

Another parameter to be determined was the event interval for the area under consideration. This should provide information on the frequency of events. The more frequently events occurred in an area, the greater its importance in interpolation. The formula describing the process of updating the event interval was presented in the equation (2).

$$I_{n+1} = I_n \cdot (1 - u) + (Ts_e - Ts_n) \cdot u \tag{2}$$

where: I_n is the current interval of the area, I_{n+1} is the new interval of the area.

For each event processed, information was also stored in the area activity table. This was then used at defined intervals to globally modify all parameters that had not been updated at that time. This was done to reduce the number of discarded events when there had been no events for a long time in the area being analysed. It also made it possible to reduce the interval values for the same areas, so that their timestamps had less impact on the result of filtering with



(a) (b)

Figure 3. The 3a shows the division of the matrix depending on the number of neighbouring regions for the event being processed, and the 3b shows an example situation for the filtering algorithm.

methods that take into account the frequency of events. This is done according to the formulas (1) and (2), but the current time (timestamp of the last processed event) is used instead of the timestamp of the processed event.

Depending on the position of the event to be processed on the matrix and the size of the regions into which it has been divided, a different number of neighbouring regions can be used for interpolation. This is illustrated in the figure 3a. The dashed line shows the boundaries of the areas into which the sensor matrix has been divided. In the blue part, four neighbours are used for the interpolation. In the green and yellow parts two neighbours are used, horizontally and vertically respective. In the red part no interpolation is necessary. The interpolation algorithm therefore works slightly differently in each of these areas.

The operation of the four neighbourhood parameter interpolation methods analysed is described below. The source code for these methods in MATLAB can be found in the github repository[1].

Figure 3b shows an example. The pixel for which the event was recorded is shown in red. The dashed lines mark the centres of the areas. The four adjacent areas are also visible. The parameters of these regions and the quantities calculated by the proposed methods are also marked.

Timestamps maximum (TM)

The first method of combining data is to perform a simple max operation. This is described by the equation (3).

$$T = max(T_{11}, T_{12}, T_{21}, T_{22}) \tag{3}$$

It uses the timestamp of each adjacent area. It is denoted by T_{11}, T_{12}, T_{21} and T_{22}. The maximum of these four plus the length of the filter is compared to the timestamp of the event being processed. If it is greater, the event is sent to the output.

[1] https : / / github . com / vision - agh / DVS _ FilterInterpolation

Bilinear interpolation (BI)

The second method involves bilinear interpolation of area timestamps depending on the position of the pixel reporting the processed event. This is described by the equations (4) and (5).

$$T_1 = \frac{T_{11}dx_2 + T_{12}dx_1}{scale} \quad T_2 = \frac{T_{21}dx_2 + T_{22}dx_1}{scale} \tag{4}$$

$$T = \frac{T_1 dy_2 + T_2 dy_1}{scale} \tag{5}$$

It also uses the timestamp of each neighbouring area. These are combined, taking into account the vertical and horizontal distance of the reporting pixel from the centres of the neighbouring areas. The result plus the length of the filter is the threshold for deciding whether an event is rejected or passed on for further processing.

Bilinear interpolation with frequency weights (BIF)

The next method is similar to the one previously discussed in the 4 subsection. However, it takes into account an additional parameter that gives an indirect indication of the number of events reported in neighbouring areas. This parameter is the average time between successive events. This is described by the equations (6).

$$\begin{aligned} T_1 &= \frac{T_{11}I_{12}dx_2 + T_{12}I_{11}dx_1}{I_{12}dx_2 + I_{11}dx_1} \\ T_2 &= \frac{T_{21}I_{22}dx_2 + T_{22}I_{21}dx_1}{I_{22}dx_2 + I_{21}dx_1} \\ T &= \frac{T_1 I_{21}I_{22}dy_2 + T_2 I_{11}I_{12}dy_1}{I_{21}I_{22}dy_2 + I_{11}I_{12}dy_1} \end{aligned} \tag{6}$$

The more events reported for an area, the more weight it has in the threshold calculation. The calculation of the number of reported events is done by estimating the time between events for each area. As with the previous methods, the result plus the length of the filter is the threshold that determines whether an event is rejected.

Distance-based interpolation with frequency weights (DIF)

The last method calculates the threshold by taking into account the distance of the reporting pixel from the centres of the neighbouring areas and, as in the previous method, the frequency of events. It uses the timestamp of each neighbouring area and its estimated interval between consecutive events. The assumption is that the weight of neighbouring areas should be proportional to the event frequency and inversely proportional to their distance from the event. Proportionality to frequency also implies inverse proportionality to the interval between events, and this quantity is calculated directly. This is described by the equations (7) and (8).

$$\begin{aligned} C_{11} &= \frac{1}{I_{11}d_{11}} \quad C_{12} = \frac{1}{I_{12}d_{12}} \\ C_{21} &= \frac{1}{I_{21}d_{21}} \quad C_{22} = \frac{1}{I_{22}d_{22}} \end{aligned} \tag{7}$$

$$T = \frac{T_{11}C_{11} + T_{12}C_{12} + T_{21}C_{21} + T_{22}C_{22}}{C_{11} + C_{12} + C_{21} + C_{22}} \tag{8}$$

The result of the above equation plus the length of the filter is the threshold that determines whether an event is rejected.

5. Evaluation

This chapter compares the proposed methods for combining neighbourhood parameters to filter event data. Several test datasets were used for this purpose.

Artificial noise

The original data was modified by adding artificial events to simulate interference in the event camera data stream. The artificial noises were randomly generated events that did not take into account movement in the scene. They were then added to the test sequence with an indication that they were artificial data. In the original data, some of the events were also noise, but it was decided to make this additional modification in order to test the effectiveness of the algorithms for noise of different intensities. It also made it easier to assess how much of the noise was removed by the proposed methods and how much of the original signal is was preserved. The first step in the analysis carried out was to assess the amount of noise in the recorded camera data sequences for a static scene. This can vary depending on the conditions of the observed scene (e.g. lighting) or the performance of the sensor. To do this, histograms of the number of events in successive time intervals were calculated. An example of a histogram for a single sequence is shown in Figure 4.

This makes it easy to distinguish the time intervals in which a falling object was recorded in the camera's field of view. The number of events is then significantly higher. To estimate the average number of events for a static scene, the highest and lowest bars were removed and the average value of the histogram patches for the remaining bars was calculated. In the example given, the 16 highest and 16 lowest values were removed and the result was 16297. Given that each interval was 0.2 s long, it can be estimated that the average noise rate for a static scene in this case was around $0.09\,\mathrm{Hz/pix}$. The estimate also shows that at least 19.87% of the recorded events were disturbances. It should be noted that this is only the average amount of noise that is not related to motion in the scene. Ideally, the events should

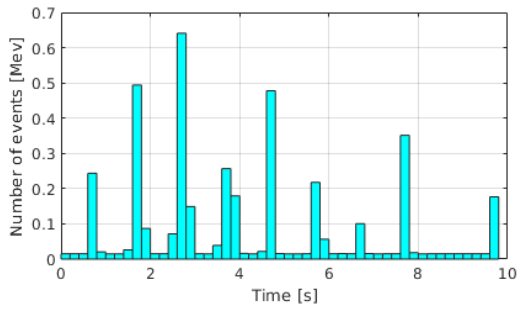


Figure 4. Example histogram of events for one of the falling corn sequences. Each bar represents the number of events in a 0.2 s time interval.

only occur in the vicinity of the falling object (assuming this is the only moving object and the lighting conditions do not change). However, there are usually a large number of events in the event stream that are not related to object movement or changes in brightness. This can be seen in Figure 1 in the pre-filtration events (left).

The proposed filtering methods were used to filter several event camera datasets. The result of the filtering was evaluated in two ways. The first way was to add artificially generated noise of varying intensity to the processed data and check the statistical results of the filtering. The quantities evaluated were the percentage of generated noise removed and the percentage of original events remaining after filtering. It should be noted, however, that some of the artificial noise may have fitted so well into the original data that it is not possible to distinguish them from the real events. It is therefore not possible to remove them all. As discussed above, the original data also contains noise. The filtering algorithm should also be able to remove this as best it can. It is therefore expected that at least some of the original events will be removed. In addition, based on the estimation of the minimum proportion of distortions in the test set presented earlier, it is possible to assume a minimum proportion of the recorded events that should be removed during filtering.

The second way was to visually assess the result of the algorithm. This involved checking the transition times of moving objects between adjacent areas to see if events were incorrectly removed. Three recordings of falling objects were used for the tests. The first two contained events recorded for falling grains of maize. These lasted 10 seconds. The third contained a recording of falling nuts lasting 6 seconds. Each of these recordings was analysed at different artificial noise intensities and for different filter lengths. For each set, the minimum interference estimation presented earlier was performed. The results of this estimation are given in the table 1. The proportion of all events that are the result of the estimation performed is given, as well as the estimated number of interferences during one

Dataset	% noises	Noise rate[Hz/pix]
I	19.87	0.09
II	18.47	0.08
III	54.05	0.12

Table 1. Estimation of the amount of noise in the datasets used

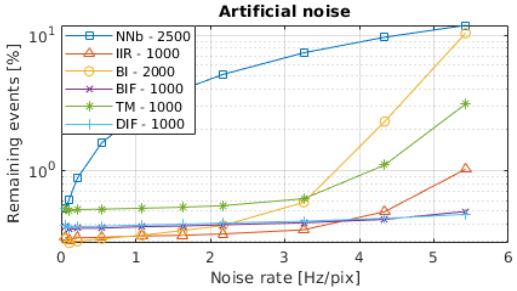


Figure 5. The amount of unremoved artificially generated artefacts by the methods analysed.

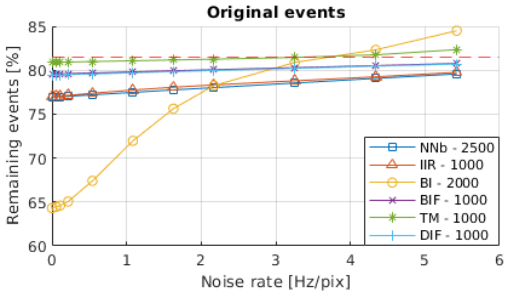


Figure 6. Number of original events left by the analysed methods under analysis.

second of sensor operation.

The performance of the proposed methods has been compared for the same filter length and different artificial noise intensities. The results for a test set are shown in the graphs 5 and 6. The first shows the % of artificial noise that has not been removed. The second shows the % of the original noise that has been passed to the output. The red dashed line shows the estimate of the maximum number of original non-interference events based on the estimation described above. For all methods based on IIR filtering, the same parameters of the considered methods were used: filter length - 1000 µs, area size - 16 × 16, update factor - 0.25.

Probably the most commonly used event data filtering algorithm, the NNb, has also been added to the comparison. This method checks whether at least one other event has been recorded in the vicinity of the processed pixel within the given time window. A time window of 2500 µs was used, and the size of the neighbourhood was 3 × 3. Its length was chosen so as not to remove too much of the correct data.

From these graphs, several conclusions can be drawn about the algorithms analysed. Firstly, the TM method passes the largest number of disturbances to the filter output. From a comparison with the estimation performed earlier, it can be seen that even for a small amount of artificial interference, this method removes a relatively small proportion of the interference. For a larger amount of noise, it removes even less than the estimation performed. The efficiency of the IIR and BI methods also decreases significantly as the amount of noise increases.

Secondly, BI removes most of the artificial noise, but the difference is quite small at around 0.2%. However, it also removes a very large proportion of the original data. This is significantly larger than the estimated minimum noise content. It can therefore be concluded that a very large proportion of the correct data is also removed. It is also worth noting that the unfiltered part of the artificial noise increases more rapidly with the amount of noise than for the other methods. An undesirable feature of this algorithm is that its effectiveness varies greatly with changes in noise intensity. In practical applications, this situation can occur very frequently (e.g. due to changes in lighting). The other proposed algorithms, in particular DIF and BIF, are very robust to noise variations.

Thirdly, the results of the two methods using weights based on event frequency are very similar. They remove the vast majority of the artificial noise and retain a fair proportion of the original events (including the estimated minimum number of noisy events). However, the first graph analysed shows that the BIF starts to lose efficiency sooner as the amount of noise increases.

Finally, the proposed methods have similar filtering efficiencies to the NNb method. However, it can be seen that the effectiveness of both frequency weighting methods scales better as the amount of interference increases. The NNb passes most of the artificially generated interference. This is due to the fact that as the interference intensity increases, the probability of two such events being very close together (both in time and space) increases, resulting in one of the events being passed to the filter output. The proposed methods are more robust in such situations.

The final element of the evaluation was to check whether the proposed algorithms achieve the stated goal of a small fraction of events removed when a moving object passes between adjacent areas. For this purpose, a performance comparison was made similar to the result of the original filtering algorithm 2b presented earlier. These are shown in figure 7. They show the result of each of the methods analysed. The same parameters were used for each of them: filter length - 400 µs, domain size - 16 × 16, update factor - 0.25. They were taken on the same moment of the same test dataset. Several conclusions can be drawn.

Firstly, Method II removes a very large proportion of correct events. This shows that BI has a negative impact on the efficiency of the algorithm. This is due to the fact that active and inactive areas affect events in the area between their centres in the same way. As a result, events that would have been passed to the output in the original version of the algorithm may be removed after using this method. The use of this method is therefore counterproductive.

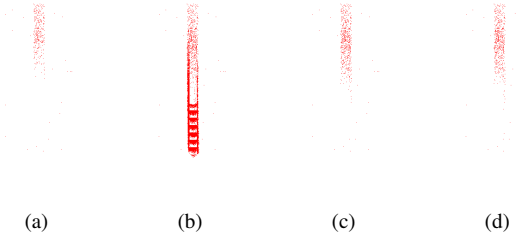


Figure 7. Filtered events for the falling corn record. 7a refers to TM, 7b to BI, 7c to BIF, and 7d to DIF.

Secondly, the result of method I shows the fewest events. It fulfils the stated requirement, i.e. it does not remove events when objects pass between neighbouring areas. On the other hand, it also marks far fewer elements that should be marked as noise. This means that it removes the least number of noisy events of the proposed methods. It is therefore less effective than the other proposed methods in terms of noise removal.

Thirdly, both method III and method IV achieve the stated objective of no removed interference when a moving object passes between adjacent areas. The result of these methods is very similar and each produces similar results. They do a good job of removing interference without removing too many valid events. For method III there are 580 events highlighted in the window shown, while for method IV there are 581. The events removed are not exactly the same. If you look closely at the images 7c and 7d you can see differences, but they are very small. Therefore, it can be concluded that both methods do a good job of removing noise and do not remove an excessive number of relevant events.

A requirement for the proposed filtering methods was also a low memory requirement. Therefore, it was calculated how much of the coefficients the proposed methods need to store in memory for a sensor with a resolution of 1280 × 720. Methods I and II require 14.85 KB of data to be stored in memory, while methods III and IV require 29.7 KB. In comparison, methods that require a timestamp to be stored for each pixel require approximately 3.67 MB of data to be stored in memory.

Recorded noise

The methodology for evaluation with recorded noise data was inspired by paper [6]. The authors proposed to record

noise data from a DVS sensor, and then insert in into a clean event stream. This methodology make it easy to distinguish the noise and signal data at the output of the evaluated filter. Moreover, they have also proposed a way to evaluate filer efficiency without using a single discrimination threshold. Instead, they have used a Receiver Operating Characteristic (ROC). It plots True Positive Rate (TPR) and False Positive Rate (FPR) across different discrimination thresholds. A measure of filtration quality is the area under the curve (AUC) calculated for the ROC plot. A higher AUC value indicates better filtration quality.

In order to apply the methodology described, several sequences containing only noise were recorded using the event camera *Prophesee EVK1 - Gen4 HD*. This involved positioning the camera so that there were no moving objects within its field of view and no changes in illumination. Several sets of data were then recorded for different levels of illumination. To further increase the range of recorded noise, the camera operating parameters responsible for sensitivity to changes in incident light intensity were changed. To test the effectiveness of the filtration in different environments and situations, it was decided to record a few test sequences. Finally, four test sequences were used. The first was a previously used sequence of falling maize grains. It is designed to simulate a situation where the system's objective is to count falling objects. Due to the small size of the objects and their high speed, this collection is very sparse. The second sequence contains a recording of people dancing in an enclosed space. It simulates the situation when a stationary camera observes a scene with a lot of movement. The third sequence contains footage of people in a closed room using a moving event camera. The fourth sequence contains data recorded while the camera is moving in an urban environment.

These were filtered to remove as much noise as possible. The previously recorded noise was then added to the filtered data. This produced a set of sequences containing known noise and the actual data. The generated sequences were then filtered using the proposed filtering methods. The AUC coefficients for these methods and the different test sequences are shown in the Table 2. In the results shown, the recorded noise with the intensity of around $1.82\,\mathrm{Hz/pix}$, was used. From the results presented, it can be noticed that the proposed methods perform very well in situations where a stationary camera captures motion in the observed scene. They perform significantly less well when processing events from a moving sensor.

6. Conclusion

The aim of the work was to propose a method of event data filtering that would have high performance but would not require a large amount of data to be stored in memory. We proposed 4 methods for event data filtering based on the method described in the paper [12]. It requires a small amount of memory, but a significant proportion of valid events are removed when an object moving in the camera's field of view passes between adjacent areas into which the matrix has been divided. The proposed algorithms consisted of interpolating data from adjacent areas so that the passage of objects between the areas would not result in the deletion of correct events. The first method consisted of finding the maximum of the timestamps of adjacent regions, the second of their bilinear interpolation, the third additionally took into account the weights of these areas based on the event frequency, and the last calculated the event distance to the centres of the neighbouring areas and the weights based on the event frequency. Their description is given in chapter 4. Chapter 5 compares their performance and memory requirements for a 1280×720 resolution sensor.

Algorithm	I	II	III	IV
NNb	0.958	0.924	0.867	0.779
IIR	0.995	0.956	0.899	0.838
BI	0.998	0.963	0.906	0.848
BIF	0.999	0.961	0.903	0.841
TM	0.998	0.950	0.881	0.816
DIF	0.999	0.960	0.901	0.842

Table 2. AUC values for the proposed filtration methods and different test scenarios

Based on the results presented, the DIF algorithm seems to perform best. However, its advantage over the BIF is small. From the evaluation carried out, it can also be concluded that the desired objective, i.e. high filtering efficiency with low memory requirements, has been achieved. In order to evaluate this, a comparison was also made between the proposed methods and probably the most commonly used method for filtering event data, i.e. NNb. The proposed methods perform better when a stationary camera observes moving objects than when the sensor itself is moving.

As part of further work, it is planned to implement the selected algorithm in an FPGA chip. This will allow a significant acceleration of the calculations performed. This implementation will be possible due to the low memory consumption of the proposed method.

7. Acknowledgements

The work presented in this paper was supported by the National Science Centre project no. 2021/41/N/ST6/03915 entitled "Acceleration of processing event-based visual data with the use of heterogeneous, reprogrammable computing devices" (first author) and the programme "Excellence initiative – research university" for the AGH University of Science and Technology.

References

[1] Juan Barrios-Avilés, Alfredo Rosado-Muñoz, Leandro D Medus, Manuel Bataller-Mompeán, and Juan F Guerrero-Martínez. Less Data Same Information for Event-Based Sensors: A Bioinspired Filtering and Data Reduction Algorithm. *Sensors*, 18(12):4122, 2018. 2

[2] Anthony Bisulco, Fernando Cladera Ojeda, Volkan Isler, and Daniel Dongyuel Lee. Near-Chip Dynamic Vision Filtering for Low-Bandwidth Pedestrian Detection. In *2020 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pages 234–239. IEEE, 2020. 2

[3] Daniel Czech and Garrick Orchard. Evaluating noise filtering for event-based asynchronous change detection image sensors. In *2016 6th IEEE International Conference on Biomedical Robotics and Biomechatronics (BioRob)*, pages 19–24. IEEE, 2016. 2

[4] Tobi Delbruck. Frame-free dynamic digital vision. In *Proceedings of Intl. Symp. on Secure-Life Electronics, Advanced Electronics for Quality Life and Society*, volume 1, pages 21–26. Citeseer, 2008. 2

[5] Guillermo Gallego, Tobi Delbrück, Garrick Orchard, Chiara Bartolozzi, Brian Taba, Andrea Censi, Stefan Leutenegger, Andrew J Davison, Jörg Conradt, Kostas Daniilidis, et al. Event-Based Vision: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(1):154–180, 2020. 1

[6] Shasha Guo and Tobi Delbruck. Low Cost and Latency Event Camera Background Activity Denoising. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1):785–795, 2023. 3, 7

[7] Shasha Guo, Ziyang Kang, Lei Wang, Shiming Li, and Weixia Xu. HashHeat: An O(C) Complexity Hashing-based Filter for Dynamic Vision Sensor. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 452–457. IEEE, 2020. 3

[8] Shriya TP Gupta, Pablo Linares-Serrano, Basabdatta Sen Bhattacharya, and Teresa Serrano-Gotarredona. Foveal-pit inspired filtering of DVS spike response. In *2021 55th Annual Conference on Information Sciences and Systems (CISS)*, pages 1–6. IEEE, 2021. 3

[9] Yuhuang Hu, Shih-Chii Liu, and Tobi Delbruck. v2e: From Video Frames to Realistic DVS Events. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1312–1321, 2021. 1

[10] Sio-Hoi Ieng, Christoph Posch, and Ryad Benosman. Asynchronous Neuromorphic Event-Driven Image Filtering. *Proceedings of the IEEE*, 102(10):1485–1499, 2014. 2

[11] Alireza Khodamoradi and Ryan Kastner. $o(n)$-Space Spatiotemporal Filter for Reducing Noise in Neuromorphic Vision Sensors. *IEEE Transactions on Emerging Topics in Computing*, 9(1):15–23, 2018. 2

[12] Marcin Kowalczyk and Tomasz Kryjak. Hardware architecture for high throughput event visual data filtering with matrix of IIR filters algorithm. In *2022 25th Euromicro Conference on Digital System Design (DSD)*, pages 284–291. IEEE, 2022. 2, 3, 8

[13] Alejandro Linares-Barranco, Francisco Gómez-Rodríguez, Vicente Villanueva, Luca Longinotti, and Tobi Delbrück. A USB3.0 FPGA event-based filtering and tracking framework for dynamic vision sensors. In *2015 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 2417–2420. IEEE, 2015. 2

[14] Hongjie Liu, Christian Brandli, Chenghan Li, Shih-Chii Liu, and Tobi Delbruck. Design of a spatiotemporal correlation filter for event-based sensors. In *2015 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 722–725. IEEE, 2015. 2

[15] Sherif AS Mohamed, Jawad N Yasin, Mohammad-Hashem Haghbayan, Jukka Heikkonen, Hannu Tenhunen, and Juha Plosila. DBA-Filter: A Dynamic Background Activity Noise Filtering Algorithm for Event Cameras. In *Intelligent Computing*, pages 685–696. Springer, 2022. 3

[16] Fernando Cladera Ojeda, Anthony Bisulco, Daniel Kepple, Volkan Isler, and Daniel D Lee. On-Device Event Filtering with Binary Neural Networks for Pedestrian Detection Using Neuromorphic Vision Sensors. In *2020 IEEE International Conference on Image Processing (ICIP)*, pages 3084–3088. IEEE, 2020. 3

[17] Vandana Padala, Arindam Basu, and Garrick Orchard. A Noise Filtering Algorithm for Event-Based Asynchronous Change Detection Image Sensors on TrueNorth and Its Implementation on TrueNorth. *Frontiers in Neuroscience*, 12:118, 2018. 2

[18] Kanglin Xiao, Xiaoxin Cui, Kefei Liu, Xiaole Cui, et al. An SNN-Based and Neuromorphic-Hardware-Implementable Noise Filter with Self-adaptive Time Window for Event-Based Vision Sensor. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2021. 3